

HW3 Report

Course: MSML606 - Spring2026

Student: Chenhongshu Yu

UID: 116610971

Implementation Details: brief explanation of how your code works

For the data structure, the hash table uses two parallel NumPy object arrays, keys and values, with a fixed capacity. Slots are initialised to EMPTY; deleted slots denoted as DELETED.

For the hash function, two methods are implemented (Division method and Multiplication method).

- Division: $h(k) = k \bmod m$
- Multiplication: $h(k) = \text{floor}(m * ((k * A) \bmod 1))$, with $A = (\text{sqrt}(5) - 1) / 2 \approx 0.6180339887$

For the collision resolution, we choose to use Linear Probing on a collision for this hash table, which the next slot is tried for “(index + 1) mod m”. We use the DELETED pattern to track, making sure that delete() function does not break existing probe chains, which deleted slots are skipped during search but reused during insert.

For the dynamic resizing, we choose to resize when load factor ≥ 0.67 (matching Python's own dict), the table doubles in size and all entries are rehashed into the new array.

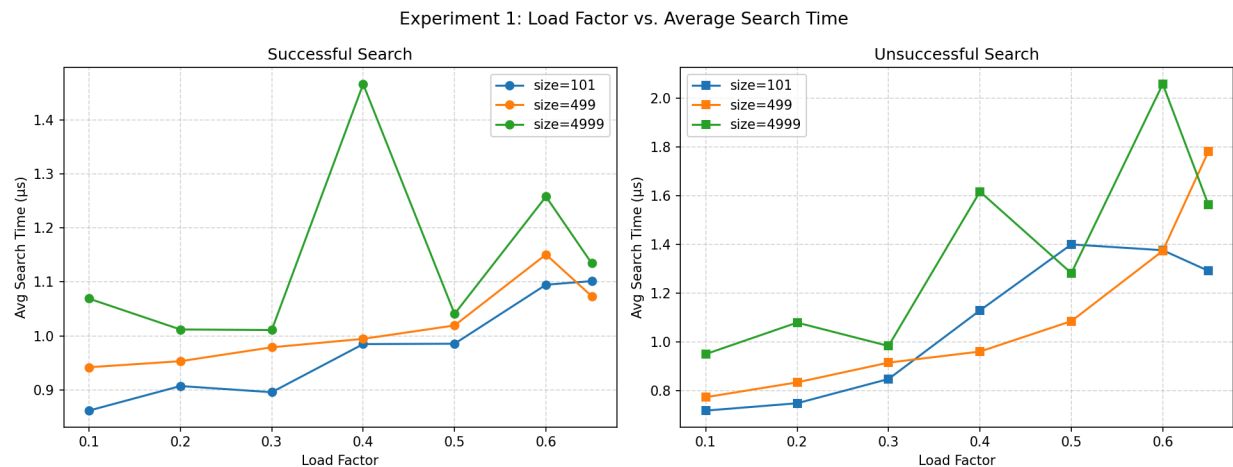
Operations:

- init(): Constructor.
- hash(key): Mentioned above (Division method and Multiplication method).
- search(key): Probe until the key is found or EMPTY hit; $O(1)$ average.
- insert(key, value): Reuse the first DELETED that has been seen, and update the value if the key exists; $O(1)$ average.
- delete(key): Mark slot as DELETED; $O(1)$ average.
- dynamicResizing(): Mentioned above (resize when load factor ≥ 0.67).

Performance Comparison

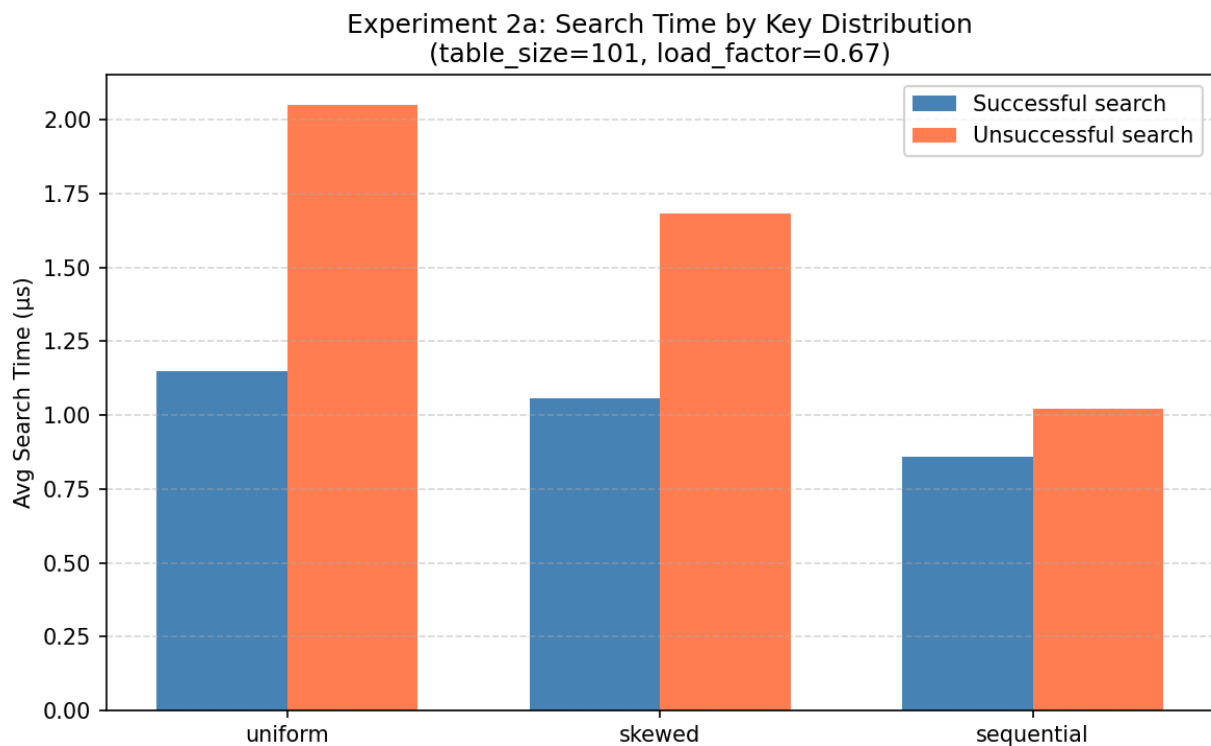
Experiment 1: Load factor vs. lookup time

- Three fixed table sizes were tested: 101, 499, 4999.
- Load factor was tested from 0.1 to 0.65.



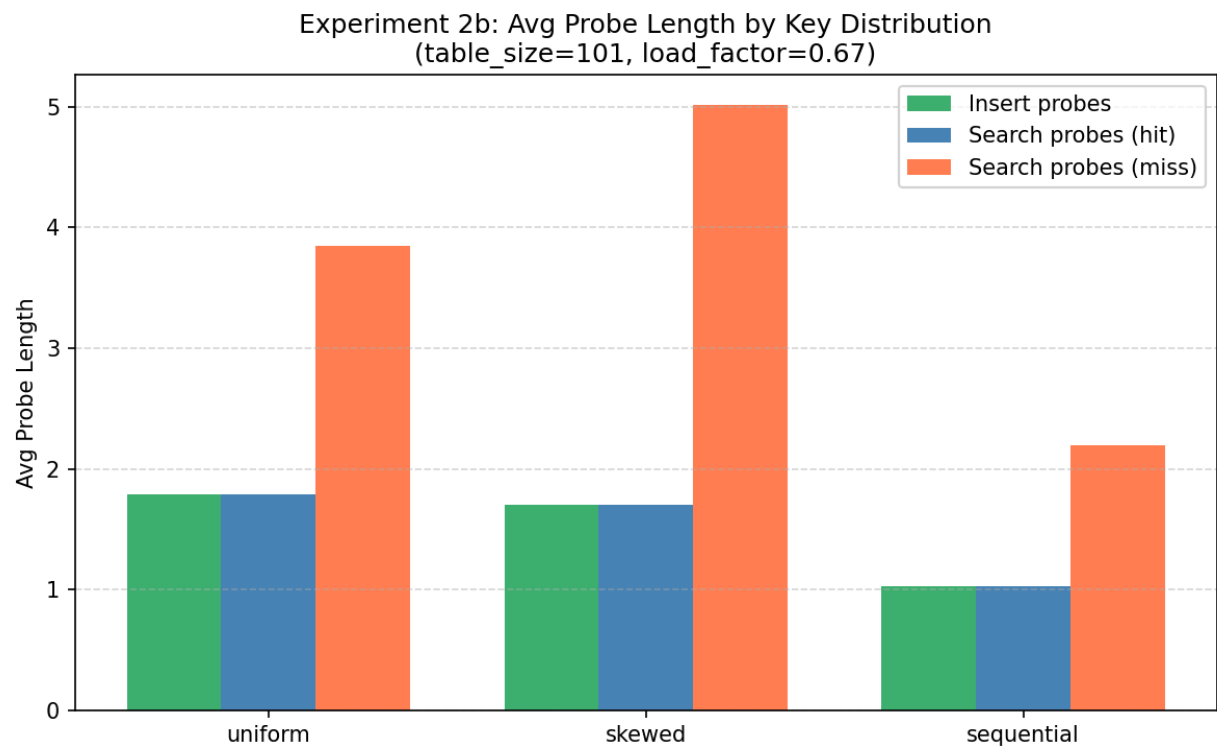
Experiment 2a: Compare search time across different distributions

- Fixed table size = 101.
- Fixed load factor = 0.67.



Distribution	Hit Time (μ s)	Miss time (μ s)
Uniform	1.1479	2.0504
Skewed	1.0586	1.6840
Sequential	0.8598	1.0205

Experiment 2b: Memory overhead analysis: Compare average probe lengths



Distribution	Avg. Insert Probes	Avg. Search Probes - HIT	Avg. Search Probes - MISS
Uniform	1.79	1.79	3.85
Skewed	1.70	1.70	5.01
Sequential	1.03	1.03	2.19

Observations & Discussion

1. Under what load factors does each method degrade? What are the memory trade-offs? Discuss both chaining and open addressing (regardless of which you chose to implement)

Answer Q1:

- For the load factor degradation, as we implemented this hash table with linear probing (open addressing) to handle the collision, we observed that the performance remains to $O(1)$ as the load factor $\lambda < 0.5$, and the performance shrinks when the load factor $\lambda = 0.7$, which creates the clustering problem because of the few empty slots left. That's why we choose the load factor $\lambda=0.67$, which is why it is triggered at this point.
 - For chaining, which is to handle the collision with chaining collision cases on each slot with a linked list. A successful search takes $O(1 + \lambda)$ average time, and an unsuccessful search also $O(1 + \lambda)$. Its performance degrades linearly with load factor rather than the steep curve seen in open addressing. That is to say, chaining tolerates higher load factors. However, chaining incurs extra memory per node as it contains multiple extra linked lists with pointers to each node within the linked list.
 - For the memory trade-off: open addressing has the contiguous array which is cache friendly, but wastes on empty slots. Also, its load factor should be less than 1, ideally as 0.67. Chaining has the linked lists with pointers to nodes, which is cache unfriendly. Also, its load factor could exceed 1, with the pointer overhead per node or entry.
2. Discuss how different distributions may influence the performance.

Answer Q2:

- Uniform: It has minimal clustering, which results in shortest probe length on average. This makes it the best choice scenario for linear probing.
- Skewed: Because it is skewed, it results in heavy collisions within a small region. It significantly causes the longer probe chain with a higher miss time.
- Sequential: It has moderate clustering, which results in middle length for the probe chain between uniform and skewed. It is the average one or the middle one when compared with the two others (uniform and skewed).